Microsoft Purview Reporting & Power BI Framework

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Purview Reporting & Power BI Framework 5

The problem . . . 5

The solution . . . 5

How it is delivered . . . 5

What it protects . . . 5

The boundary . . . 5

The bottom line . . . 6

1. Solution Overview — Microsoft Purview Reporting & Power BI Framework 6

What this solution is . . . 6

Why this matters . . . 6

How it works . . . 7

The reporting sources (each licensing-gated — verify your tenant) . . . 7

Naming convention (used in every doc and script) . . . 7

The standard deployment model . . . 8

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

2. Requirements Guide — Microsoft Purview Reporting & Power BI Framework 9

2.1 Compliance licensing for the in-scope reporting sources (GR1) . . . 9

2.2 Least-privilege Purview reporting roles (GR2) . . . 9

2.3 Security & Compliance PowerShell + Microsoft Graph toolchain (GR3) . . . 9

2.4 Graph permission scopes (consented, read-only) . . . 9

2.5 Power BI licensing and a workspace path . . . 10

2.6 An export storage location and the dataset contract . . . 10

2.7 Privacy & role separation for HR-sensitive sources . . . 10

2.8 Compliance, records, privacy & legal stakeholders and a pilot scope (GR5, GR6) . . . 10

Checklist . . . 11

Global Requirements — Microsoft Purview Solutions 11

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 11

2. Purview roles (least-privilege) and portal access . . . 12

3. Security & Compliance PowerShell + Graph toolchain . . . 12

4. Data sources & scope in scope . . . 12

5. Compliance, records, privacy & legal stakeholders . . . 12

6. A pilot scope and change control . . . 13

3. Implementation Guide — Microsoft Purview Reporting & Power BI Framework 13

Stage 0 — Assess (read-only discovery) . . . 13

Stage 1 — Design the dataset contract . . . 14
Stage 2 — Configure the export pipeline and pilot workspace . . . 14
Stage 3 — Pilot (validate every number against the portal) . . . 15
Stage 4 — Validate the pipeline health . . . 16
Stage 5 — Expand to production with a scheduled refresh . . . 16
Post-implementation . . . 16
5. Testing & Validation — Microsoft Purview Reporting & Power BI Framework 17
Test matrix . . . 17
Reconciliation method (T5–T8) . . . 18
Acceptance criteria (all must hold) . . . 18
What testing does NOT cover (deferred by design) . . . 19
6. Troubleshooting — Microsoft Purview Reporting & Power BI Framework 19
`OPSREPORT-Discovery.ps1` marks a source "not licensed" or "access denied" . . . 19
An export table is empty when you expect data . . . 19
Dashboard figures don't match the portal (T5–T8) . . . 20
`Export-ActivityExplorerData` / `Search-UnifiedAuditLog` "term is not recognized" . . . 20
The export set is stale / pipeline health is red . . . 20
`-CreateWorkspace` does nothing / MicrosoftPowerBIMgmt missing . . . 20
Scheduled refresh registered but the dataset in Power BI doesn't update . . . 20
A change was applied without `-WhatIf` review . . . 21
The RiskAggregate export contains subject or content fields (privacy, T11) . . . 21
When to escalate beyond this solution . . . 21
7. Best Practices — Microsoft Purview Reporting & Power BI Framework 21
Data lineage & defensibility . . . 21
Read-only & least privilege . . . 22
Privacy-by-design (HR-sensitive sources) . . . 22
Pilot-before-broad (the reporting equivalent of simulate-first) . . . 22
Freshness & reliability . . . 22
Licensing & tenant reality . . . 23
Operations . . . 23
8. Deliverables Summary — Microsoft Purview Reporting & Power BI Framework 23
Documents . . . 23
PowerShell tools (5) . . . 24
Named artifacts this solution creates . . . 25
Definition of done . . . 25
Explicitly out of scope (deferred) . . . 25

Appendix A — PowerShell Scripts 26

Executive Summary — Microsoft Purview Reporting & Power BI Framework

The problem

Microsoft Purview holds a rich compliance picture — data-loss events, sensitivity- and retention-label activity, audit trails, alerts, and posture scores — but that picture is trapped inside the Purview portal, one capability at a time, behind admin roles and licensing. Leadership almost never opens the portal, so the compliance posture never reaches the people accountable for it. The result is a real gap: the data exists, but the organization cannot show, at a glance, that its controls are working — a weakness in any audit, board review, or regulatory conversation.

The solution

A repeatable **reporting framework** that turns Purview's scattered signal into a small set of **Power BI dashboards a leader actually reads** — refreshed on a schedule, with a documented, defensible data lineage. It pulls data **read-only** from Microsoft Graph and Security & Compliance PowerShell into a governed, contract-shaped export set, then binds Power BI dashboards to that set: one page per capability (DLP & labels, retention & records, audit activity, compliance posture, and *aggregate-only* risk) plus an executive summary. It uses only Microsoft-native tooling — no third-party product.

How it is delivered

The framework follows a disciplined model — **Assess** → **Design** → **Configure** → **Pilot (validate)** → **Validate** → **Expand**. Dashboards are built and **reconciled against the portal's own numbers in a pilot workspace before any leadership audience sees them**, then promoted to production with a scheduled refresh. Five PowerShell tools automate discovery, the data pipeline, validation, and pipeline health — all safe-by-default (read-only, with writes gated behind an explicit switch, and no stored credentials).

What it protects

- **Privacy-by-design.** Insider-risk and communication-compliance data is reported as **aggregate counts only** — never case content or named subjects — with strict role separation.
- **Defensibility.** Every number has a named source, a refresh cadence, and a reconciliation record — the answer to "how do you know."
- **Honesty about licensing.** Purview reporting sources are plan-gated (many require E5 / E5 Compliance) and some are preview; the framework verifies each source in your tenant and labels any it cannot license, rather than inventing a figure.

The boundary

This solution delivers the **reporting capability and the evidence** behind each number. It does **not** make the **regulatory, records, or legal determinations** — what to retain, what a signal means, whether a posture is sufficient. Those remain with the organization's compliance, records, privacy, HR, and legal

functions.

The bottom line

Reporting leaders actually read — the Purview compliance posture, delivered as validated, scheduled, defensible dashboards, at a fraction of the cost of a bespoke consulting engagement.

1. Solution Overview — Microsoft Purview Reporting & Power BI Framework

What this solution is

Microsoft Purview generates a great deal of compliance signal — DLP policy matches, sensitivity- and retention-label activity, audit events, alerts, and compliance/secure-score posture — but that signal lives inside the Purview portal's built-in reports, one workload at a time, behind licensing and role gates. Leadership rarely opens the portal, so the picture never reaches the people who own the risk. This solution is a repeatable **reporting framework**: it defines a governed set of Purview data exports (pulled from **Microsoft Graph** and **Security & Compliance PowerShell**), a consistent dataset shape, and a **Power BI** dashboard model built on top of that data — so the compliance posture becomes a small number of dashboards a leader actually reads, refreshed on a schedule, with a documented, defensible data lineage. It uses only Microsoft-native tooling (Purview portal, Graph, Security & Compliance PowerShell, Power BI) — no third-party product.

> ***Boundary.*** *This solution delivers the* ***reporting capability*** *— the pipeline, dataset, and dashboards — and the evidence behind each number. It does* ***not*** *make the underlying* ***regulatory, records-retention, or legal determinations*** *(what to retain, what a DLP match means legally, whether an insider-risk signal warrants action). Those stay with the organization's* ***compliance, records, privacy, HR, and legal*** *functions. Purview capabilities are heavily* ***licensing-/plan-dependent*** *(many reporting sources require* ***Microsoft 365 E5*** *or the* ***E5 Compliance*** *add-on; some are in preview) and the portal is evolving — every data source below is flagged* ***"verify your licensing/tenant"*** *rather than assumed.*

Why this matters

- **Portal reports don't reach leadership.** Each Purview report is scoped to one capability and one admin audience; there is no native cross-capability, leadership-ready view. The gap between "the data exists" and "the board saw it" is itself a compliance and data-risk exposure.
- **A trend beats a snapshot.** A single portal screenshot is a point in time; a governed, scheduled dataset lets you show *direction* — DLP matches falling after a policy tightened, label coverage rising, audit activity within expected bounds.
- **Defensibility.** When an auditor or regulator asks "how do you know," a reporting framework with a documented source (which Graph endpoint / which cmdlet), a refresh cadence, and a validation gate is an answer; an ad-hoc export is not.
- **Privacy discipline.** Insider-risk and communication-compliance data is HR/privacy-sensitive.

This framework reports **aggregate counts only** for those sources (never case content or named subjects) and keeps role separation intact — reporting operators never gain case access.

How it works

1. **Scope** is defined first: which Purview capabilities are in scope (DLP, sensitivity labels,

retention/records, audit, alerts, secure/compliance score, and *aggregate-only* insider-risk / communication-compliance counts), which workloads (Exchange, SharePoint, OneDrive, Teams, endpoints), and the reporting date window.

1. **Data is pulled read-only** from two sources: **Microsoft Graph** (security alerts, secure score,

audit-log queries) and **Security & Compliance PowerShell** (`Export-ActivityExplorerData` for label/DLP activity, `Get-ProtectionAlert`, `Search-UnifiedAuditLog`). Each pull writes to a consistent, dated export set — the framework's single source of truth.

1. **The exports are shaped into a stable dataset** (a fixed set of columns per source) so the Power BI

model does not break when a portal report changes layout. The dataset is the contract between the pipeline and the dashboards.

1. **Power BI dashboards** are built on the dataset — one page per capability plus an executive summary

page — first in a **pilot workspace**, validated against the portal's own numbers, then promoted to a production workspace with a scheduled refresh.

1. **The pipeline is health-checked** on each run (connections succeed, exports are fresh, row counts

are sane) so a silent failure never produces a stale-but-plausible dashboard.

The reporting sources (each licensing-gated — verify your tenant)

Source	Pulled via	Typical licensing
DLP & label activity (Activity Explorer)	`Export-ActivityExplorerData` (S&C PowerShell)	E5 / E5 Compliance — verify
Purview alerts (DLP, IP, etc.)	`Get-ProtectionAlert` (S&C PowerShell) / Graph `security/alerts_v2`	E3 baseline, E5 for advanced — verify
Audit activity	`Search-UnifiedAuditLog` / Graph `security/auditLog/queries`	Audit Standard (E3) / Premium (E5) — verify
Compliance / secure-score posture	Graph `security/secureScores`	Verify per tenant
Insider risk / comm. compliance	**aggregate counts only** (portal export)	E5 / E5 Compliance — verify

Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, Purview portal, and Power BI.

Artifact	Name
Reporting scope definition	`Reporting Scope – Purview – Config`
Dated export set	`Purview Data Export – AllSources – Report`
Export storage location	`Purview Exports – <yyyyMM> – Report`
Dashboard dataset	`Purview Compliance Dataset – Report`
Pilot Power BI workspace	`Purview Reporting – Pilot – Stage1`
Production Power BI workspace	`Purview Reporting – Prod – Stage2`
Scheduled refresh	`Purview Refresh – Daily – Recurring`
Pipeline health log	`Purview Pipeline Health – Report`

The standard deployment model

> *Assess the available sources and licensing → design the dataset and dashboard pages → configure the exports and build the pilot workspace →* ***pilot in a test workspace and validate every number against the portal*** *→ expand to the production workspace with a scheduled refresh. Pattern:* ***Assess → Design → Configure → Pilot (validate) → Validate → Expand.***

What you can control

- Which Purview capabilities and workloads are in scope, and the reporting date window.
- The dataset shape (which columns each source contributes) and the dashboard page set.
- The pilot audience that reviews the dashboards before production promotion.
- The refresh cadence (e.g. daily) and who receives the leadership view.
- The privacy posture for HR-sensitive sources (aggregate-only, role-separated).

Limitations (be honest with buyers)

- **Licensing gates the data, not the framework.** If a source (e.g. Activity Explorer) is not licensed in your tenant, that dashboard page has no data until the licensing is in place — the framework flags it rather than inventing numbers.
- **Latency.** Audit and Activity Explorer data has ingestion latency (often hours to a day); a dashboard reflects data up to the source's own delay, not real time.
- **Portal parity.** Portal reports and PowerShell/Graph exports can differ slightly in how they aggregate; the validation gate documents the reconciliation so a small variance is explained, not a surprise.
- **Insider-risk / communication-compliance content is out of scope by design** — only aggregate

counts are reported, to preserve privacy and role separation.

2. Requirements Guide — Microsoft Purview Reporting & Power BI Framework

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (GR1–GR6); this guide adds the solution-specific ones. Purview reporting sources are **licensing-/plan-dependent** and some are in preview — where a source is gated, verify availability in *your* tenant rather than assuming it.

2.1 Compliance licensing for the in-scope reporting sources (GR1)

Description: Microsoft 365 licensing that exposes each source you intend to report on — E3 for baseline audit/alerts; **E5 / E5 Compliance** for Activity Explorer (DLP & label activity), advanced alerts, Audit Premium, and DSPM/insider-risk aggregates. **Why:** Every dashboard page is only as available as its source; an unlicensed source returns no data. **Verify:** In the Purview portal confirm each capability's report/page loads (e.g. **Activity Explorer**, **Alerts**, **Audit**); in the M365 admin center confirm the E5/E5 Compliance add-on. **Implement:** Assign/purchase the required plan before building the page that depends on it; until then, mark that page "not licensed — verify tenant" rather than populating it.

2.2 Least-privilege Purview reporting roles (GR2)

Description: A reporting account holding the **read** roles the sources need — typically **Compliance Administrator** or **Compliance Data Administrator** (or the granular **View-Only** report roles), plus **Audit Reader / Audit Manager** for the audit log and **Security Reader** for Graph security data. Global Administrator is **not** required for read-only reporting. **Why:** Purview uses fine-grained role groups; each export cmdlet/endpoint checks the matching role. **Verify:** Purview portal → **Roles & scopes** → **Permissions** shows the role group membership; `Search-UnifiedAuditLog` and `Export-ActivityExplorerData` run without an access error. **Implement:** Assign the least-privilege role groups under change control; keep the reporting identity separate from any insider-risk / communication-compliance case role (see 2.7).

2.3 Security & Compliance PowerShell + Microsoft Graph toolchain (GR3)

Description: An admin workstation with **ExchangeOnlineManagement** (provides `Connect-IPPSSession` and `Search-UnifiedAuditLog`) and **Microsoft.Graph** PowerShell SDK, using modern-auth / MFA. **Why:** All five scripts pull data through `Connect-IPPSSession` (S&C PowerShell) and `Connect-MgGraph` (Graph). **Verify:** `Connect-IPPSSession` and `Connect-MgGraph` both succeed with the reporting account; `Get-Module -ListAvailable ExchangeOnlineManagement, Microsoft.Graph.Authentication`. **Implement:** `Install-Module ExchangeOnlineManagement, Microsoft.Graph -Scope CurrentUser`.

2.4 Graph permission scopes (consented, read-only)

Description: Delegated read scopes for the Graph sources: `SecurityEvents.Read.All` (alerts), `SecurityActions.Read.All` / `SecureScore` via `SecurityEvents.Read.All`, and `AuditLog.Read.All` where the Graph audit-query API is used. No write scope is needed for reporting. **Why:** The framework is read-only against Graph; these scopes pull alerts, secure score, and audit data without granting any change capability. **Verify:** Run `OPSREPORT-Discovery.ps1` (read-only) — it connects and reports which scopes are consented and which are missing. **Implement:** Consent at first `Connect-MgGraph`; if admin consent is required, an admin approves the read scopes once in Entra (**Identity** → **Applications** → **Enterprise applications**).

2.5 Power BI licensing and a workspace path

Description: Power BI Desktop (free) to build the model, and **Power BI service** capacity to host it — a **Pro** licence for pilot/small audiences or **Premium/Fabric capacity** (PPU/F-SKU) for scheduled refresh at scale — plus the ability to create the pilot and production workspaces. **Why:** The dashboards are Power BI; scheduled refresh and sharing require the service and the right capacity. **Verify:** You can create a workspace in the Power BI service and publish a report to it; a Pro/PPU licence is assigned to the publishing account. **Implement:** Assign Power BI Pro/PPU (or provision Premium/Fabric capacity); create the `Purview Reporting – Pilot – Stage1` workspace. The optional `MicrosoftPowerBIMgmt` module lets `OPSREPORT-Configuration.ps1` create the workspace for you.

2.6 An export storage location and the dataset contract

Description: A governed location for the exports (a secured folder, SharePoint document library, or storage account) that Power BI can read, and agreement on the **dataset shape** (the fixed columns each source contributes) recorded as `Purview Compliance Dataset – Report`. **Why:** The exports are the single source of truth the dashboards bind to; a stable column contract is what keeps the model from breaking when a portal report layout changes. **Verify:** The location exists, the reporting identity can write to it, and Power BI can read from it; the dataset column list is documented. **Implement:** Provision the location with least-privilege access; record the dataset contract before building dashboards (the implementation guide specifies the columns per source).

2.7 Privacy & role separation for HR-sensitive sources

Description: For insider-risk and communication-compliance sources, a decision that only **aggregate counts** (e.g. number of alerts by severity) are reported — never case content, subjects, or reviewers — and that the reporting identity has **no** insider-risk/communication-compliance case role. **Why:** These capabilities are HR/privacy-sensitive; reporting must preserve privacy-by-design and strict role separation, with outcomes deferred to HR/legal. **Verify:** The reporting account is not a member of any insider-risk or communication-compliance role group; only aggregate counts are configured for those pages. **Implement:** Keep the roles separated; where an aggregate is needed, export the count only (portal aggregate export), not the underlying records.

2.8 Compliance, records, privacy & legal stakeholders and a pilot scope (GR5, GR6)

Description: Named compliance/records/privacy/legal stakeholders who own what each number *means*, a **pilot audience** for the dashboards, and a change window to promote to production. **Why:** The

framework reports the data; the interpretation and any action are the stakeholders'. A pilot validates accuracy before leadership sees it. **Verify:** Stakeholders and a pilot audience are named; the pilot workspace is agreed as the first target. **Implement:** Identify owners and the pilot audience before publishing; agree the promotion path to production.

Checklist

- Compliance licensing confirmed for each in-scope source (E5/E5 Compliance where required) — GR1
- Least-privilege reporting roles assigned (Compliance/Audit/Security Reader) — GR2
- ExchangeOnlineManagement + Microsoft.Graph installed; `Connect-IPPSSession` / `Connect-MgGraph` succeed — GR3
- Graph read scopes consented (`SecurityEvents.Read.All`, `AuditLog.Read.All`)
- Power BI Desktop + Pro/PPU (or Premium/Fabric capacity); pilot workspace creatable
- Export storage location provisioned; `Purview Compliance Dataset – Report` contract recorded
- Insider-risk / comm-compliance = aggregate-only; reporting identity role-separated
- Stakeholders + pilot audience named; promotion change window agreed — GR5/GR6

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

• **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

• **Why required:** Configuration, automation, and the provided scripts run through these.

• **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.

• **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

• **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams, endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

• **Why required:** Purview policies key off the in-scope workloads and the classification scheme.

• **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with stakeholders.

• **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

• **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

• **Why required:** Purview configuration must align to enterprise policy and legal/records requirements, not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate → Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Microsoft Purview Reporting & Power BI Framework

This is the exact, ordered build. It follows the house model **Assess** → **Design** → **Configure** → **Pilot (validate)** → **Validate** → **Expand**. Every artifact is named per the convention in the overview and reused verbatim. Reporting is **read-only** against Purview/Graph; the only writes are creating your export storage, Power BI workspaces, and a refresh schedule — those are gated behind `-Commit` in the scripts and behind an explicit change window here. **Build and validate in the pilot workspace before any leadership audience sees a number.** Each stage has a **pass/fail gate** and a **rollback**.

> *Purview portal paths and cmdlets below are current as of authoring; the portal evolves and sources are licensing-gated — **verify each path and each source's availability in your tenant**. Do not populate a page whose source is not licensed; mark it "not licensed — verify tenant".*

Stage 0 — Assess (read-only discovery)

0.1 On the admin workstation, connect read-only:

```
Connect-IPPSSession
Connect-MgGraph -Scopes 'SecurityEvents.Read.All','AuditLog.Read.All'
```

0.2 Run `OPSREPORT-Discovery.ps1`. It inventories, without changing anything:

- which reporting sources are reachable for the reporting identity (Activity Explorer via `Export-ActivityExplorerData`, alerts via `Get-ProtectionAlert`, audit via `Search-UnifiedAuditLog`, secure score via Graph `security/secureScores`);
- which Graph scopes are consented vs missing;
- a licensing/role readiness note per source (available / not licensed / access denied).

0.3 Run `OPSREPORT-Set-ScopeHealth.ps1 -ShowScope` to record the intended **reporting scope** — in-scope capabilities, workloads (Exchange, SharePoint, OneDrive, Teams, endpoints), and the reporting date window — as `Reporting Scope – Purview – Config`.

Gate 0 (pass/fail): PASS when every intended source reports **available** (or is explicitly marked *not licensed — verify tenant*), scopes are consented, and the scope definition is recorded. FAIL → resolve licensing/roles (Requirements 2.1–2.4) before proceeding. **Rollback:** none — read-only.

Stage 1 — Design the dataset contract

1.1 Record the **dataset contract** `Purview Compliance Dataset – Report`: the fixed columns each source contributes, so the Power BI model binds to a stable shape. Author it exactly as:

Source table	Key columns
`DLP_LabelActivity` (Activity Explorer)	`Date, Workload, Activity, SensitivityLabel, RetentionLabel, DlpPolicy, Count`
`Alerts` (`Get-ProtectionAlert` / Graph)	`CreatedTime, PolicyName, Category, Severity, Status, Count`
`AuditActivity` (`Search-UnifiedAuditLog`)	`Date, Workload, Operation, RecordType, Count`
`Posture` (secure score)	`Date, ControlCategory, CurrentScore, MaxScore, Percent`
`RiskAggregate` (aggregate-only)	`Date, Source, Severity, Count` *(counts only — no subjects/content)*

1.2 Design the **dashboard pages** (one per capability + an executive page): `Executive Summary`, `DLP & Labels`, `Retention & Records`, `Audit Activity`, `Compliance Posture`, `Risk (Aggregate)`. Record the intended visual per page (trend line, severity bar, coverage gauge).

Gate 1: PASS when the column contract and page list are documented and reviewed with the compliance stakeholder (Requirement 2.8). **Rollback:** none — design only.

Stage 2 — Configure the export pipeline and pilot workspace

This stage writes. Run each configuration step with `-WhatIf` first (the default), review the planned action, then re-run with `-Commit` inside the agreed change window.

2.1 Export storage. Create the storage location and the dated export set:

```
.\OPSREPORT-Configuration.ps1 -CreateStorage -StoragePath '<secured path or library>' -WhatIf
.\OPSREPORT-Configuration.ps1 -CreateStorage -StoragePath '<secured path or library>' -Commit
```

This provisions `Purview Exports – <yyyyMM> – Report` under least-privilege access.

2.2 Pilot Power BI workspace. Create `Purview Reporting – Pilot – Stage1`:

```
.\OPSREPORT-Configuration.ps1 -CreateWorkspace -WorkspaceName 'Purview Reporting - Pilot - Stage1' -WhatIf
.\OPSREPORT-Configuration.ps1 -CreateWorkspace -WorkspaceName 'Purview Reporting - Pilot - Stage1' -Commit
```

(Uses `MicrosoftPowerBIMgmt`; if absent, create the workspace in the Power BI service and skip `-CreateWorkspace`.)

2.3 First data pull. Generate the export set into storage (read-only against Purview/Graph):

```
.\OPSREPORT-Report.ps1 -StoragePath '<...>' -StartDate (Get-Date).AddDays(-30) -EndDate (Get-Date)
```

This produces `Purview Data Export – AllSources – Report` — one CSV/JSON per source table matching the dataset contract.

2.4 Build the model in Power BI Desktop. Connect Power BI Desktop to the export set, load each source table with the contracted columns, set relationships on `Date`, and build the six pages from Stage 1.2. Publish to `Purview Reporting – Pilot – Stage1`.

Gate 2: PASS when the export set exists with all licensed source tables populated, and the pilot report renders all six pages. **Rollback:** delete the pilot workspace and the export set (no production impact — nothing is shared yet).

Stage 3 — Pilot (validate every number against the portal)

This is the mandatory validation ring — the reporting equivalent of "pilot in simulation before enforce." **No leadership audience yet.**

3.1 Run `OPSREPORT-Validation.ps1 -StoragePath '<...>'` (read-only). It reconciles each dashboard figure against its Purview source for the same window:

- DLP/label activity counts vs Activity Explorer in the portal;
- alert counts by severity vs the Purview **Alerts** page;
- audit operation counts vs a matching `Search-UnifiedAuditLog` query;
- secure-score percentages vs the Graph `security/secureScores` value.

3.2 Record any variance and its explanation (aggregation/latency differences are expected and documented, not silently accepted).

3.3 Have the pilot audience (Requirement 2.8) review the dashboards for clarity and correctness.

Gate 3 (pass/fail): PASS when every figure reconciles to its portal source within the documented tolerance and the pilot audience signs off. FAIL → correct the export/model (usually a scope, date window, or column mapping issue) and re-validate. **Rollback:** the dashboards remain pilot-only; nothing is promoted until this gate passes.

Stage 4 — Validate the pipeline health

4.1 Run `OPSREPORT-Set-ScopeHealth.ps1 -HealthCheck -StoragePath '<...>'`. It confirms: connections succeed, the export set is **fresh** (within the expected age), each source table has a sane row count, and no source silently returned empty when it should have data. Output is logged as `Purview Pipeline Health – Report`.

Gate 4: PASS when the health check is green (fresh, populated, connections OK). FAIL → fix the failing source before scheduling a refresh (a scheduled stale dashboard is worse than none).

Stage 5 — Expand to production with a scheduled refresh

Writes — perform inside the change window; `-WhatIf` first.

5.1 Production workspace. Create `Purview Reporting – Prod – Stage2`:

```
.\OPSREPORT-Configuration.ps1 -CreateWorkspace -WorkspaceName 'Purview Reporting - Prod - Stage2' -WhatIf
.\OPSREPORT-Configuration.ps1 -CreateWorkspace -WorkspaceName 'Purview Reporting - Prod - Stage2' -Commit
```

5.2 Promote the validated report from pilot to production (publish the same `.pbix` to the prod workspace, or use a deployment pipeline if on Premium/Fabric).

5.3 Schedule the data pull + refresh. Configure the daily export + dataset refresh `Purview Refresh – Daily – Recurring`:

```
.\OPSREPORT-Configuration.ps1 -ScheduleRefresh -Cadence Daily -WhatIf
.\OPSREPORT-Configuration.ps1 -ScheduleRefresh -Cadence Daily -Commit
```

(A scheduled task runs `OPSREPORT-Report.ps1` to refresh the export set, then Power BI service refreshes the dataset from it.)

5.4 Grant the leadership audience read access to the production workspace/app (least-privilege viewer role).

Gate 5 (pass/fail): PASS when production renders all pages from the scheduled export, the refresh runs on cadence, and the audience can view it. FAIL → roll back to pilot-only. **Rollback:** remove audience access, disable `Purview Refresh – Daily – Recurring`, and revert to the pilot workspace; production changes are additive (a new workspace + schedule), so removing them restores the prior state with no data loss.

Post-implementation

- The recurring value is the **cadence**: the export runs daily, the dataset refreshes, and leadership

reads a current trend — re-run `OPSREPORT-Validation.ps1` periodically to keep the reconciliation evidence current.

- Keep the **dataset contract** authoritative: if a new source is added or a portal report changes,

update `Purview Compliance Dataset – Report` first, then the export and model — never let the model drift from the contract.

- Interpretation and any action on the numbers remain with compliance/records/privacy/legal.

5. Testing & Validation — Microsoft Purview Reporting & Power BI Framework

Testing proves two things: the pipeline pulls the **right** data, and the dashboards show it **accurately**. Every test is read-only against Purview/Graph. Run the gates in order; a solution is "done" only when every gate passes and the reconciliation evidence is recorded. Tolerances exist because portal aggregation and export aggregation can differ slightly by latency and grouping — a variance is acceptable **only when explained**, never when unexplained.

Test matrix

#	Test	How	Pass	Fail action
T1	Source reachability	`OPSREPORT-Discovery.ps1`	Every intended source = available (or marked *not licensed — verify tenant*); scopes consented	Fix licensing/roles (Req 2.1–2.4)
T2	Scope recorded	`OPSREPORT-Set-ScopeHealth.ps1 -ShowScope`	`Reporting Scope – Purview – Config` lists capabilities, workloads, date window	Define scope before building
T3	Export set produced	`OPSREPORT-Report.ps1 -StartDate ... -EndDate ...`	One table per licensed source, columns match the dataset contract	Fix scope/date/column mapping
T4	Dataset contract conformance	Inspect export headers vs `Purview Compliance Dataset – Report`	Every table has exactly the contracted columns	Correct the export shaping
T5	DLP/label reconciliation	`OPSREPORT-Validation.ps1` vs Activity Explorer	Counts match portal within tolerance	Investigate scope/window mismatch
T6	Alert reconciliation	Validation vs Purview **Alerts** page	Counts by severity match	Check policy scope / time filter

#	Test	How	Pass	Fail action
T7	Audit reconciliation	Validation vs matching `Search-UnifiedAuditLog`	Operation counts match	Align RecordType/date filters
T8	Posture reconciliation	Validation vs Graph `security/secureScores`	Score % matches the source value	Confirm the score date pulled
T9	Pipeline health	`OPSREPORT-Set-ScopeHealth.ps1 -HealthCheck`	Fresh, populated, connections OK → `Purview Pipeline Health – Report` green	Fix the failing source before scheduling
T10	Pilot render	Open pilot report	All six pages render with data	Rebuild the failing page/model
T11	Privacy check	Inspect `RiskAggregate` export	Counts only — no subjects, no case content	Remove any non-aggregate field immediately
T12	Scheduled refresh	After Stage 5, check next-day run	Export refreshed on cadence; dataset refreshed; dashboards current	Re-check the scheduled task / capacity refresh

Reconciliation method (T5–T8)

For each figure, run the dashboard for a fixed window (e.g. last 7 days) and pull the **same** window from the portal source. Record: dashboard value, portal value, variance, and the reason for any variance (latency, grouping, scope). File this as the reporting **evidence** — it is the answer to "how do you know these numbers are right."

Acceptance criteria (all must hold)

- T1–T4: pipeline produces a contract-conformant export set for every licensed source.
- T5–T8: every dashboard figure reconciles to its portal source within the documented tolerance, with variances explained.
- T9: pipeline health is green (fresh, populated, connected).

- T10: all six dashboard pages render.
- T11: HR-sensitive sources are **aggregate-only** — verified, no subject/content fields present.
- T12: the scheduled refresh keeps production current.

What testing does NOT cover (deferred by design)

- **Whether a number is "good."** Testing proves the number is *accurate*, not whether the DLP rate, retention coverage, or risk count is acceptable — that judgment is the compliance/records/privacy/ legal stakeholders'.
- **Regulatory sufficiency.** Whether the reported posture satisfies a regulation is a legal/compliance determination, not a test result here.
- **Insider-risk / communication-compliance case detail** — out of scope; only aggregate counts are tested for privacy conformance.

6. Troubleshooting — Microsoft Purview Reporting & Power BI Framework

The failure modes you'll hit building and running the reporting pipeline. Each: **symptom** → **likely cause** → **fix.** Most "the dashboard is wrong" reports are a scope/date/window mismatch, a lapsed license on a source, or a stale export — not a real data fault. Reporting is **read-only** against Purview/Graph (`Connect-IPPSSession`, `Connect-MgGraph`); the only writes are storage/workspace/schedule, all gated.

`OPSREPORT-Discovery.ps1` marks a source "not licensed" or "access denied"

- **Cause A — licensing.** Activity Explorer / Audit (Premium) / secure score are licensing-gated (E5 / E5 Compliance). **Fix:** confirm the entitlement (Requirements 2.1–2.4); until then mark the page **"not licensed — verify tenant"** and do not populate it. **Do not invent a source.**
- **Cause B — missing Graph scope.** `security/secureScores` needs `SecurityEvents.Read.All`; audit needs `AuditLog.Read.All`. **Fix:** re-consent `Connect-MgGraph -Scopes ...`; re-run discovery.
- **Cause C — role.** The reporting identity lacks the read role. **Fix:** grant the least-privilege reader role for that source.

An export table is empty when you expect data

- **Cause A — window/scope.** The date window or workload scope excludes the activity. **Fix:** confirm `Reporting Scope – Purview – Config` (`-ShowScope`) and the `-StartDate`/`-EndDate` on `OPSREPORT-Report.ps1`.

• **Cause B — latency.** Activity Explorer / audit data has ingestion latency; a very recent window may

be sparse. **Fix:** widen the window or allow ingestion time.

• **Cause C — lapsed license.** A source that worked now returns empty. **Fix:** the pipeline-health

note flags this — verify the source's licensing hasn't lapsed (Requirement 2.1).

Dashboard figures don't match the portal (T5–T8)

• **Cause — aggregation/latency/grouping differences.** Portal aggregation and export aggregation can

differ slightly by grouping and refresh timing. **Fix:** this is expected **only when explained** — run `OPSREPORT-Validation.ps1`, record the dashboard value, portal value, variance, and reason (latency/grouping/scope) as the reconciliation evidence. An **unexplained** variance is a real mismatch: align the RecordType/date/scope filters to the portal query and re-validate.

`Export-ActivityExplorerData` / `Search-UnifiedAuditLog` "term is not recognized"

• **Cause:** wrong endpoint/module. `Export-ActivityExplorerData`, `Get-ProtectionAlert` are **S&C

PowerShell (`Connect-IPPSSession`); `Search-UnifiedAuditLog` is Exchange Online (`Connect-ExchangeOnline`); secure score is Graph (`Connect-MgGraph`). Fix:** connect the right endpoint for each source before running the report.

The export set is stale / pipeline health is red

• **Cause:** the scheduled export didn't run, or ran under an account that couldn't reach a source.

Fix: `OPSREPORT-Set-ScopeHealth.ps1 -HealthCheck` shows the age/row-count per table; check the `Purview Refresh – Daily – Recurring` scheduled task ran and its account has the read roles. **A scheduled stale dashboard is worse than none** — fix the source before relying on it.

`-CreateWorkspace` does nothing / MicrosoftPowerBIMgmt missing

• **Cause:** the `MicrosoftPowerBIMgmt` module isn't installed, or you're not connected. **Fix:**

`Install-Module MicrosoftPowerBIMgmt` then `Connect-PowerBIServiceAccount`, or create the workspace in the Power BI service and **skip** `-CreateWorkspace`. The framework doesn't require the module — it's a convenience.

Scheduled refresh registered but the dataset in Power BI doesn't update

• **Cause:** two refreshes are involved — the **export set** (local scheduled task running

`OPSREPORT-Report.ps1`) and the **Power BI dataset** (refreshed by the Power BI service from the export). Registering the task only handles the first. **Fix:** configure the dataset's scheduled refresh (and gateway, if the source is on-prem) in the Power BI service so it pulls the refreshed export.

A change was applied without `-WhatIf` review

- **Cause:** `-Commit` was passed before reviewing the dry-run. **Fix:** all writes are additive —

delete the export folder/workspace or `Unregister-ScheduledTask "Purview Refresh – Daily – Recurring"` to roll back; then re-run with `-WhatIf` first, review, and `-Commit`.

The RiskAggregate export contains subject or content fields (privacy, T11)

- **Cause:** a non-aggregate field leaked into the risk source. **Fix (immediate):** remove the

offending field — insider-risk/communication-compliance sources are **aggregate-only** (counts, no subjects, no case content) by design. Re-run T11 before anyone sees the page. Case detail is out of scope; only aggregate counts are reported.

When to escalate beyond this solution

- **Whether a number is "good" (DLP rate, retention coverage, risk count acceptable)** → **compliance / records / privacy / legal** (this solution proves the number is *accurate*, not *acceptable* — deferred).
- **Whether the reported posture satisfies a regulation** → a **legal/compliance** determination, not a reporting output.
- **A source genuinely unavailable despite correct licensing** → verify against current Microsoft documentation and open a Microsoft support case; Purview reporting surfaces and Graph endpoints evolve.

7. Best Practices — Microsoft Purview Reporting & Power BI Framework

Field-tested practices for a Purview reporting framework that stays accurate, defensible, and safe. Grouped by theme; each is actionable.

Data lineage & defensibility

- **Treat the dataset contract as law.** `Purview Compliance Dataset – Report` is the single agreement

between the exports and the dashboards. Change the contract *first*, then the export, then the model — never let the Power BI model reach around the contract to a raw source.

- **Every number needs a source you can name.** For each dashboard figure, be able to say which cmdlet

or Graph endpoint produced it and for what window. That sentence is your audit answer.

- **Keep the reconciliation evidence current.** Re-run `OPSREPORT-Validation.ps1` on a cadence and

retain the output; a dashboard nobody has reconciled in six months is not defensible.

• **Version the model.** Keep the `.pbix` in source control (or a Fabric deployment pipeline) so a dashboard change is reviewable and revertible.

Read-only & least privilege

• **Report with a read-only identity.** The reporting account needs *View-Only* / *Reader* roles, not Compliance Administrator write access and never Global Administrator. Reporting should not be able to change a policy.

• **Separate the reporting identity from case roles.** The account that builds insider-risk / communication-compliance aggregates must hold **no** case role — role separation is the privacy control.

• **Scope Graph consent to read.** Consent only `*.Read.All` scopes; there is no reason a reporting pipeline holds a write scope.

Privacy-by-design (HR-sensitive sources)

• **Aggregate-only, always.** For insider risk and communication compliance, export counts by severity/date — never subjects, reviewers, or case content. If a field could identify a person under investigation, it does not belong in a report dataset.

• **Pseudonymize where a breakdown is needed.** If leadership needs a by-department cut, aggregate to the department, not the individual.

• **Defer the outcome.** The report shows *that* signals exist and their volume; whether any warrants action is an HR/legal decision, stated as such on the dashboard.

Pilot-before-broad (the reporting equivalent of simulate-first)

• **Never publish an unreconciled dashboard to leadership.** The pilot workspace exists so wrong numbers are caught before the board sees them. A reporting error erodes trust as badly as a policy error.

• **Reconcile against the portal, not against yourself.** Validation compares your export to Purview's own numbers — the portal is the reference of record.

• **Explain every variance.** A small difference between export and portal is fine when documented (latency, grouping); an unexplained difference is a defect.

Freshness & reliability

• **A stale dashboard is worse than none.** Health-check freshness on every run; if the export didn't refresh, surface it on the dashboard (a "data as of" stamp) rather than showing yesterday's number as today's.

• **Fail loud.** If a source returns empty when it should have data, the pipeline flags it — an empty chart must never be indistinguishable from "no incidents."

• **Watch licensing drift.** If an E5 add-on lapses, a source silently goes empty; the health check catches it and marks the page *not licensed — verify tenant*.

Licensing & tenant reality

• **Verify each source in *your* tenant.** Purview features are plan-gated and some are preview; confirm availability before promising a page, and label unavailable pages honestly.

• **Don't invent a source.** If a capability isn't licensed, the page is empty and labelled — never synthesize or estimate a compliance number.

Operations

• **Least-privilege the audience too.** Grant leadership *Viewer* on the app/workspace, not contributor.

• **Keep the export set retained per policy.** The dated exports are records; retain and dispose of them per the organization's records schedule (a records determination — defer to records management).

• **Document the runbook.** Who owns the pipeline, what the daily job does, and how to recover a failed refresh (see `ADMIN_DOCS`/troubleshooting).

8. Deliverables Summary — Microsoft Purview Reporting & Power BI Framework

Everything included in this solution, and what "done" looks like.

Documents

Document	Purpose
01 · Solution Overview	What the reporting framework is, the sources, the naming convention, the deployment model
02 · Requirements Guide	Solution-specific prerequisites (licensing, roles, toolchain, Power BI, storage, privacy) with verify/implement steps, referencing category GR1–GR6
03 · Implementation Guide	The ordered build — Assess → Design → Configure → Pilot (validate) → Validate → Expand — with named artifacts, pass/fail gates, and rollback
05 · Testing & Validation	The T1–T12 gate matrix and the portal reconciliation method

Document	Purpose
06 · Troubleshooting	Symptom → cause → resolution for the common pipeline, licensing, and refresh failures
07 · Best Practices	Lineage, least-privilege, privacy-by-design, pilot-before-broad, freshness, licensing reality
Executive Summary	One-page business framing for decision-makers

PowerShell tools (5)

All are safe-by-default: read-only against Purview/Graph, with writes (storage, workspaces, schedule) gated behind `-Commit` (default `-WhatIf`). No credentials or secrets are stored in code; auth is interactive modern-auth (`Connect-IPPSSession` / `Connect-MgGraph`).

Script	Role	Default	Purpose
`OPSREPORT-Discovery.ps1`	Discovery	Read-only	Inventory reachable reporting sources, consented scopes, and per-source licensing/role readiness
`OPSREPORT-Configuration.ps1`	Configuration/Remediation	`-WhatIf`	Create export storage, pilot/production Power BI workspaces, and the daily refresh schedule (writes only with `-Commit`)
`OPSREPORT-Report.ps1`	Reporting	Read-only export	Pull all in-scope sources into the contract-conformant export set (`Purview Data Export –AllSources –Report`)
`OPSREPORT-Validation.ps1`	Validation	Read-only	Reconcile each dashboard figure against its Purview/Graph source for the same window

Script	Role	Default	Purpose
`OPSREPORT-Set-ScopeHealth.ps1`	Scope/health helper	Read-only	Record the reporting scope and run the pipeline health check (freshness, row counts, connections)

Named artifacts this solution creates

- `Reporting Scope – Purview – Config` — the in-scope capabilities, workloads, and date window
- `Purview Compliance Dataset – Report` — the dataset column contract
- `Purview Data Export – AllSources – Report` / `Purview Exports – <yyyyMM> – Report` — the export set + storage
- `Purview Reporting – Pilot – Stage1` / `Purview Reporting – Prod – Stage2` — the Power BI workspaces
- `Purview Refresh – Daily – Recurring` — the scheduled export + dataset refresh
- `Purview Pipeline Health – Report` — the health-check log

Definition of done

- Every intended source is **available** (or explicitly marked *not licensed — verify tenant*).
- The export set is **contract-conformant** and produced on schedule.
- Every dashboard figure **reconciles** to its portal source within a documented tolerance.
- HR-sensitive sources are **aggregate-only**, with role separation verified.
- The **pilot** was validated and signed off before production; production refreshes on cadence and the leadership audience can view it.
- Interpretation and any action on the numbers remain with compliance/records/privacy/legal.

Explicitly out of scope (deferred)

- Regulatory, records-retention, and legal **determinations** (what the numbers mean or require).
- Insider-risk / communication-compliance **case content** — only aggregate counts are reported.
- Any change to Purview **policies** — this framework reports; it does not configure the underlying compliance controls.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OPSREPORT-Configuration.ps1

Purpose: The only write-path in the framework, and it is gated. -CreateStorage provisions the dated export set folder Purview Exports - <yyyyMM> - Report under a secured path; -CreateWorkspace creates a Power BI workspace (New-PowerBIWorkspace via MicrosoftPowerBIMgmt); -ScheduleRefresh registers a local scheduled task Purview Refresh - Daily - Recurring that re-runs OPSREPORT-Report.ps1 to refresh the export set. SAFE BY DEFAULT: every action runs in -WhatIf and prints the exact planned change; -Commit is required to apply. It never reads or writes Purview/Graph data itself. HOW TO RUN Export storage (dry-run then commit): .\OPSREPORT-Configuration.ps1 -CreateStorage -StoragePath 'D:\PurviewExports' -WhatIf .\OPSREPORT-Configuration.ps1 -CreateStorage -StoragePath 'D:\PurviewExports' -Commit Power BI workspace (needs MicrosoftPowerBIMgmt + Connect-PowerBIServiceAccount): .\OPSREPORT-Configuration.ps1 -CreateWorkspace -WorkspaceName 'Purview Reporting - Pilot - Stage1' -Commit Daily refresh task that runs OPSREPORT-Report.ps1: .\OPSREPORT-Configuration.ps1 -ScheduleRefresh -Cadence Daily -StoragePath 'D:\PurviewExports' -ReportScript '.\OPSREPORT-Report.ps1' -Commit EXPECTED RESULTS -WhatIf: the planned folder / workspace / scheduled-task is printed; nothing created. -Commit: the dated export folder exists, and/or the workspace is created, and/or the scheduled task is registered. Rollback: delete the folder/workspace or Unregister-ScheduledTask (all additive). .NOTES Purview — Reporting & Power BI Framework. -WhatIf by default. Power BI workspace creation uses MicrosoftPowerBIMgmt; if absent, create the workspace in the Power BI service and skip -CreateWorkspace. The scheduled task runs under the account you register it with — grant least privilege. No secrets in code. No cmdlets invented. > [CmdletBinding()] param([switch]$CreateStorage, [switch]$CreateWorkspace, [switch]$ScheduleRefresh,

How to run: Export storage (dry-run then commit): .\OPSREPORT-Configuration.ps1 -CreateStorage -StoragePath 'D:\PurviewExports' -WhatIf .\OPSREPORT-Configuration.ps1 -CreateStorage -StoragePath 'D:\PurviewExports' -Commit Power BI workspace (needs MicrosoftPowerBIMgmt + Connect-PowerBIServiceAccount): .\OPSREPORT-Configuration.ps1 -CreateWorkspace -WorkspaceName 'Purview Reporting - Pilot - Stage1' -Commit Daily refresh task that runs OPSREPORT-Report.ps1: .\OPSREPORT-Configuration.ps1 -ScheduleRefresh -Cadence Daily -StoragePath 'D:\PurviewExports' -ReportScript '.\OPSREPORT-Report.ps1' -Commit EXPECTED RESULTS -WhatIf: the planned folder / workspace / scheduled-task is printed; nothing created. -Commit: the dated export folder exists, and/or the workspace is created, and/or the scheduled task is registered. Rollback: delete the folder/workspace or Unregister-ScheduledTask (all additive). .NOTES Purview — Reporting & Power BI Framework. -WhatIf by default. Power BI workspace creation uses MicrosoftPowerBIMgmt; if absent, create the workspace in the Power BI service and skip -CreateWorkspace. The scheduled task runs under the account you register it with — grant least privilege. No secrets in code. No cmdlets invented. > [CmdletBinding()] param([switch]$CreateStorage, [switch]$CreateWorkspace, [switch]$ScheduleRefresh,

OPSREPORT-Discovery.ps1

Purpose: Inventories which Purview reporting data sources are reachable for the current reporting identity — Activity Explorer (Export-ActivityExplorerData), Purview alerts (Get-ProtectionAlert), the unified audit log

(Search-UnifiedAuditLog), and secure-score posture (Graph security/secureScores) — and reports per-source readiness (available / not licensed / access denied) plus which Graph read scopes are consented. Makes no changes. HOW TO RUN Connect first, then run: Connect-IPPSSession Connect-MgGraph -Scopes 'SecurityEvents.Read.All','AuditLog.Read.All' .\OPSREPORT-Discovery.ps1 PowerShell 5.1+/7+. Requires ExchangeOnlineManagement and Microsoft.Graph modules and a least-privilege reporting role (Compliance/Audit/Security Reader). Purview sources are licensing-gated — an unavailable source is reported, never assumed. EXPECTED RESULTS A per-source readiness table. PASS = every intended source is 'available' (or explicitly 'not licensed - verify tenant') and required scopes are consented. .NOTES Purview — Reporting & Power BI Framework. Read-only. No secrets in code; interactive modern-auth only. > [CmdletBinding()] param([string]$CsvPath)

How to run: Connect first, then run: Connect-IPPSSession Connect-MgGraph -Scopes 'SecurityEvents.Read.All','AuditLog.Read.All' .\OPSREPORT-Discovery.ps1 PowerShell 5.1+/7+. Requires ExchangeOnlineManagement and Microsoft.Graph modules and a least-privilege reporting role (Compliance/Audit/Security Reader). Purview sources are licensing-gated — an unavailable source is reported, never assumed. EXPECTED RESULTS A per-source readiness table. PASS = every intended source is 'available' (or explicitly 'not licensed - verify tenant') and required scopes are consented. .NOTES Purview — Reporting & Power BI Framework. Read-only. No secrets in code; interactive modern-auth only. > [CmdletBinding()] param([string]$CsvPath)

OPSREPORT-Report.ps1

Purpose: Pulls each licensed Purview reporting source for the requested window and writes one contract-conformant table per source into the export set 'Purview Data Export - AllSources - Report' (the dataset Power BI binds to). Sources: DLP/label activity (Export-ActivityExplorerData), alerts (Get-ProtectionAlert), audit activity (Search-UnifiedAuditLog), and posture (Graph security/secureScores). Read-only against Purview/Graph — the only writes are the local export files. Aggregate-only for HR-sensitive sources; never exports case content or subjects. HOW TO RUN Connect-IPPSSession ; Connect-MgGraph -Scopes 'SecurityEvents.Read.All','AuditLog.Read.All' .\OPSREPORT-Report.ps1 -StoragePath C:\PurviewExports -StartDate (Get-Date).AddDays(-30) -EndDate (Get-Date) PowerShell 5.1+/7+. A source that is not licensed is skipped with a note, never fabricated. EXPECTED RESULTS CSV files under <StoragePath>\Purview Exports - <yyyyMM> - Report: DLP_LabelActivity.csv, Alerts.csv, AuditActivity.csv, Posture.csv — columns matching 'Purview Compliance Dataset - Report'. A console summary of rows per table. .NOTES Purview — Reporting & Power BI Framework. Read-only source pulls. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$StoragePath, [datetime]$StartDate = (Get-Date).AddDays(-30), [datetime]$EndDate = (Get-Date))

How to run: Connect-IPPSSession ; Connect-MgGraph -Scopes 'SecurityEvents.Read.All','AuditLog.Read.All' .\OPSREPORT-Report.ps1 -StoragePath C:\PurviewExports -StartDate (Get-Date).AddDays(-30) -EndDate (Get-Date) PowerShell 5.1+/7+. A source that is not licensed is skipped with a note, never fabricated. EXPECTED RESULTS CSV files under <StoragePath>\Purview Exports - <yyyyMM> - Report: DLP_LabelActivity.csv, Alerts.csv, AuditActivity.csv, Posture.csv — columns matching 'Purview Compliance Dataset - Report'. A console summary of rows per table. .NOTES Purview — Reporting & Power BI Framework. Read-only source pulls. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$StoragePath, [datetime]$StartDate = (Get-Date).AddDays(-30), [datetime]$EndDate = (Get-Date))

OPSREPORT-Set-ScopeHealth.ps1

Purpose: Two read-only helpers for the Purview reporting framework: (1) -ShowScope records/reads the reporting scope 'Reporting Scope - Purview - Config' (in-scope capabilities, workloads, date window) to a JSON file; (2) -HealthCheck inspects the export set and confirms it is fresh (within MaxAgeHours), populated (row counts sane), and that expected source tables are present — logging 'Purview Pipeline Health - Report'. Never changes Purview or Graph. HOW TO RUN .\OPSREPORT-Set-ScopeHealth.ps1 -ShowScope -ScopeFile C:\PurviewExports\scope.json .\OPSREPORT-Set-ScopeHealth.ps1 -HealthCheck -StoragePath C:\PurviewExports -MaxAgeHours 26 PowerShell 5.1+/7+. Read-only; safe to run any time. EXPECTED RESULTS -ShowScope: the recorded scope printed. -HealthCheck: a per-table freshness/row-count table and a green/red health verdict. PASS = fresh, populated, all expected tables present. .NOTES Purview — Reporting & Power BI Framework. Read-only. No secrets in code. > [CmdletBinding(DefaultParameterSetName='Health')] param([Parameter(ParameterSetName='Scope')][switch]$ShowScope, [Parameter(ParameterSetName='Scope')][string]$ScopeFile = './scope.json', [Parameter(ParameterSetName='Scope')][string[]]$Capabilities = @('DLP','SensitivityLabels','Retention','Audit','Posture','RiskAggregate'), [Parameter(ParameterSetName='Scope')][string[]]$Workloads = @('Exchange','SharePoint','OneDrive','Teams','Endpoint'), [Parameter(ParameterSetName='Scope')][int]$WindowDays = 30,

How to run: .\OPSREPORT-Set-ScopeHealth.ps1 -ShowScope -ScopeFile C:\PurviewExports\scope.json .\OPSREPORT-Set-ScopeHealth.ps1 -HealthCheck -StoragePath C:\PurviewExports -MaxAgeHours 26 PowerShell 5.1+/7+. Read-only; safe to run any time. EXPECTED RESULTS -ShowScope: the recorded scope printed. -HealthCheck: a per-table freshness/row-count table and a green/red health verdict. PASS = fresh, populated, all expected tables present. .NOTES Purview — Reporting & Power BI Framework. Read-only. No secrets in code. > [CmdletBinding(DefaultParameterSetName='Health')] param([Parameter(ParameterSetName='Scope')][switch]$ShowScope, [Parameter(ParameterSetName='Scope')][string]$ScopeFile = './scope.json', [Parameter(ParameterSetName='Scope')][string[]]$Capabilities = @('DLP','SensitivityLabels','Retention','Audit','Posture','RiskAggregate'), [Parameter(ParameterSetName='Scope')][string[]]$Workloads = @('Exchange','SharePoint','OneDrive','Teams','Endpoint'), [Parameter(ParameterSetName='Scope')][int]$WindowDays = 30,

OPSREPORT-Validation.ps1

Purpose: For a fixed window, re-pulls each source live and compares the count to the exported table in the export set, so every dashboard figure can be reconciled to Purview's own numbers. Reports variance per source with a PASS/FAIL against a tolerance. Read-only — changes nothing. Also confirms the export set conforms to the dataset column contract and that HR-sensitive tables are aggregate-only. HOW TO RUN Connect-IPPSSession ; Connect-MgGraph -Scopes 'SecurityEvents.Read.All','AuditLog.Read.All' .\OPSREPORT-Validation.ps1 -StoragePath C:\PurviewExports -Days 7 -TolerancePercent 5 PowerShell 5.1+/7+. A documented variance (latency/grouping) is acceptable; an unexplained one is a defect. EXPECTED RESULTS A reconciliation table (source | exported | live | variance% | result) and a contract/privacy conformance check. PASS = every source within tolerance and contract-conformant. .NOTES Purview — Reporting & Power BI Framework. Read-only reconciliation. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$StoragePath, [int]$Days = 7, [double]$TolerancePercent = 5)

How to run: Connect-IPPSSession ; Connect-MgGraph -Scopes 'SecurityEvents.Read.All','AuditLog.Read.All' .\OPSREPORT-Validation.ps1 -StoragePath C:\PurviewExports -Days 7 -TolerancePercent 5 PowerShell 5.1+/7+. A documented variance (latency/grouping) is acceptable; an unexplained one is a defect. EXPECTED RESULTS A reconciliation table (source | exported | live | variance% | result) and a contract/privacy conformance check. PASS = every source within tolerance and contract-conformant. .NOTES Purview — Reporting & Power BI Framework. Read-only reconciliation. No secrets in code. > [CmdletBinding()] param([Parameter(Mandatory)][string]$StoragePath, [int]$Days = 7, [double]$TolerancePercent = 5)